

Bash Scripting Cheat Sheet

Write small shell scripts: variables, tests, loops and functions.

Using it

```
#!/usr/bin/env bash ← first line: run this file with bash (then chmod +x it, and run ./script.sh)
set -euo pipefail ← stop on any error (-e), on unset variables (-u), on failed pipes
name="world"; echo "Hello, $name" ← no spaces around = ; use it with $, inside double quotes
```

Variables, quoting and special values

Write	Means	Write	Means
"\$var"	Value of var (quotes keep spaces safe)	\$1 \$2 ...	Arguments given to the script/function
'\$var'	Literally \$var: single quotes do nothing	"\$@"	All arguments, each kept separate
\$(cmd)	The output of cmd (today=\$(date +%F))	#	Number of arguments
\$(x + 1)	Whole-number maths	?	Exit status of the last command (0 = OK)
{var:-def}	var, or def if var is empty/unset	\$\$	This script's process ID
{var%.txt}	var with .txt cut off the end	\$0	The script's own name

Tests: if [[...]]

Test	True when	Test	True when
-f file	file exists and is a normal file	\$a == \$b	Strings are equal (!= = not equal)
-d dir	dir exists and is a folder	\$s == *.log	String matches a wildcard pattern
-e path	Path exists (any type)	\$s =~ ^[0-9]+\$	Matches a regex
-r -w -x	Readable / writable / runnable	\$a -eq \$b	Numbers equal (-ne = not equal)
-s file	file exists and isn't empty	-lt -gt	Less / greater than (-le -ge : or =)
-z "\$s"	String is empty (-n = not empty)	&& !	And / or / not, inside [[]]

Control flow and functions

```
if [[ -f "$f" ]]; then echo "file"
elif [[ -d "$f" ]]; then echo "folder"
else echo "missing"; exit 1 ← exit 1 = failed
fi
case "$1" in
  start|up) do_start ;; ← |= or
  stop)    do_stop  ;;
  *)       echo "usage: $0 start|stop" ;;
esac
```

```
for f in *.log; do gzip "$f"; done ← each file
for i in {1..5}; do echo "$i"; done
while read -r line; do ← each line of a file
  echo "$line"
done < hosts.txt
greet() { ← define a function
  local who="$1"; echo "hi $who"
}
greet "Mark" ← call it like a command
```

Reading the output: bash -x

```
$ bash -x ./backup.sh /data ← -x shows each command as it runs (or put set -x in the script)
+ src=/data ← += a command bash ran, with variables already filled in
++ date +%F ← ++ = a command inside $( ), one level deeper
+ echo 'done 2026-10-03'
done 2026-10-03 ← no += the script's real output
```

Handy recipes

Command	Does
[[\$# -eq 1]] { echo "usage: \$0 file"; exit 1; }	Usage message unless given exactly 1 argument
for f in *.jpg; do mv "\$f" "\${f%.jpg}.jpeg"; done	Renames every .jpg to .jpeg
if ping -c1 -W1 "\$h" > /dev/null; then ...	Uses a command's success as the test
trap 'rm -f "\$tmp"' EXIT	Cleans up a temp file however the script ends

Remember: quote every variable (**"\$var"**) so spaces in values don't break things. Spaces go inside test brackets: **[[-f "\$f"]]**, not **[[-f \$f]]**.